

Paxos Explained from Scratch

Hein Meling and Leander Jehl



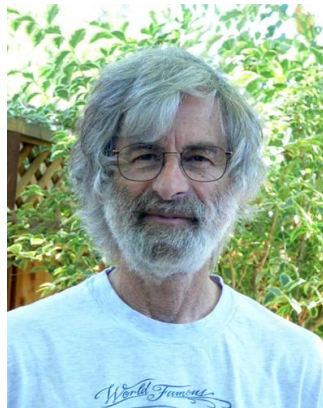
University of
Stavanger

hein.meling@uis.no

DAT520 Distributed Systems 2026

Leslie Lamport

- ▶ Microsoft Research
- ▶ Many important contributions to distributed computing theory
- ▶ 2013 Turing Award winner
- ▶ But most know for $\text{\LaTeX}$



The Part-Time Parliament

LESLIE LAMPORT
Digital Equipment Corporation

Recent archaeological discoveries on the island of Paxos reveal that the parliament functioned despite the peripatetic propensity of its part-time legislators. The legislators maintained consistent copies of the parliamentary record, despite their frequent forays from the chamber and the forgetfulness of their messengers. The Paxos parliament's protocol provides a new way of implementing the state-machine approach to the design of distributed systems.

Categories and Subject Descriptors: C2.4 [Computer-Communications Networks]: Distributed Systems—*Network operating systems*; D4.5 [Operating Systems]: Reliability—*Fault-tolerance*; J.1 [Administrative Data Processing]: Government

General Terms: Design, Reliability

Additional Key Words and Phrases: State machines, three-phase commit, voting

Vertical Paxos and Primary-Backup Replication

Leslie Lamport, Dahlia Malkhi, Lidong Zhou
Microsoft Research

9 February 2009
corrected 26 August 2009

The Paxos Register

Harry C. Li, Allen Clement, Amitanand S. Aiyer, and Lorenzo Alvisi
The University of Texas at Austin
Department of Computer Sciences
{harry, aclement, anand, lorenzo}@cs.utexas.edu

There Is More Consensus in Egalitarian Parliaments

Iulian Moraru, David G. Andersen, Michael Kaminsky
Carnegie Mellon University and Intel Labs

Cheap Paxos

Leslie Lamport and Mike Massa

MICROSOFT

Fast Paxos

Leslie Lamport

Paxos for System Builders

Jonathan Kirsch and Yair Amir

Paxos Made Live - An Engineering Perspective

Tushar Chandra
Robert Griesemer
Joshua Redstone

June 20, 2007

Paxos Made Moderately Complex Paxos Made Simple

Robbert van Renesse
Cornell University
rvr@cs.cornell.edu

Leslie Lamport

March 25, 2011

01 Nov 2001

In Search of an Understandable Consensus Algorithm

Diego Ongaro and John Ousterhout
Stanford University

(Draft of April 7, 2013, under submission to SOSP)

When You Don't Trust Clients: Byzantine Proposer Fast Paxos

Hein Meling*, Keith Marzullo†, and Alessandro Mei‡

*Department of Electrical Engineering and Computer Science, University of Stavanger, Norway

†Department of Computer Science and Engineering, University of California San Diego, USA

‡Department of Computer Science, Sapienza University of Rome, Italy

Email: hein.meling@uis.no, marzullo@cs.ucsd.edu, mei@di.uniroma1.it

What is Paxos and why is it Relevant?

- ▶ Fault tolerant consensus protocol

What is Paxos and why is it Relevant?

- ▶ Fault tolerant consensus protocol
- ▶ Used to order client requests sent to a fault tolerant server
 - ▶ For example a fault tolerant resource manager

What is Paxos and why is it Relevant?

- ▶ Fault tolerant consensus protocol
- ▶ Used to order client requests sent to a fault tolerant server
 - ▶ For example a fault tolerant resource manager
- ▶ Used in production systems: Chubby, ZooKeeper, and Spanner

What is Paxos and why is it Relevant?

- ▶ Fault tolerant consensus protocol
- ▶ Used to order client requests sent to a fault tolerant server
 - ▶ For example a fault tolerant resource manager
- ▶ Used in production systems: Chubby, ZooKeeper, and Spanner
- ▶ It is always safe

Objectives and Approach

- ▶ Explain Paxos
 - ▶ In a step-wise manner
 - ▶ With minimal changes in each step

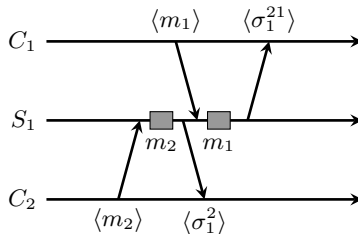
Objectives and Approach

- ▶ Explain Paxos
 - ▶ In a step-wise manner
 - ▶ With minimal changes in each step
- ▶ Objective
 - ▶ Understand why it works and why the solution is necessary
 - ▶ (not how to implement or formally prove it)

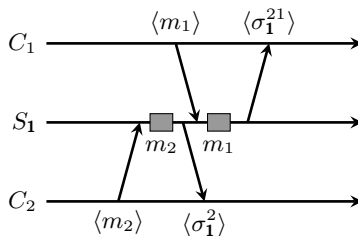
Objectives and Approach

- ▶ Explain Paxos
 - ▶ In a step-wise manner
 - ▶ With minimal changes in each step
- ▶ Objective
 - ▶ Understand why it works and why the solution is necessary
 - ▶ (not how to implement or formally prove it)
- ▶ Approach
 - ▶ Starting from a simple client/server system
 - ▶ Build fault tolerant server (replicated state machine)
 - ▶ Construct Multi-Paxos
 - ▶ Decompose Multi-Paxos into Paxos

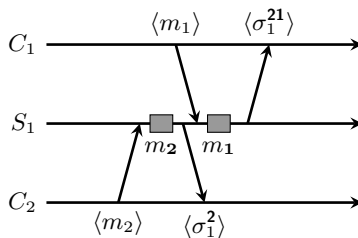
A Stateful Service: *SingleServer*



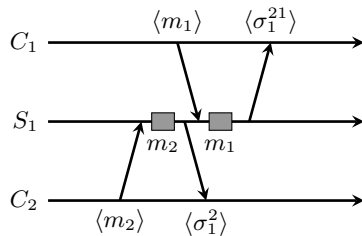
A Stateful Service: *SingleServer* (Subscript)



A Stateful Service: *SingleServer* (Superscript)



A Stateful Service: *SingleServer*



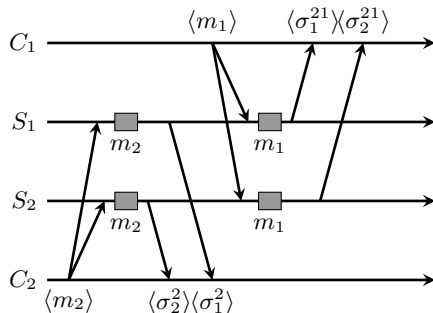
- ▶ Client C_2 sees: σ^2
- ▶ Client C_1 sees: σ^{21}
 - ▶ σ^2 is a prefix of σ^{21}
- ▶ Corresponds to execution sequence: m_2m_1

We Want to Make the Service Fault
Tolerant!

Assumptions

- ▶ Asynchronous environment: No bounds on
 - ▶ Processing delay
 - ▶ Communication delay
 - ▶ Clock drift
 - ▶ Unreliable failure detectors
- ▶ Unreliable communication
 - ▶ May take arbitrarily long to deliver msg
 - ▶ Msgs can be duplicated and lost
- ▶ Deterministic operations
- ▶ Processes may crash

Fault Tolerance with Two Servers

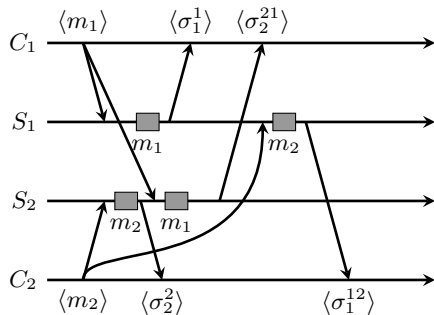


- ▶ Client C_2 sees: σ^2
- ▶ Client C_1 sees: σ^{21}
 - ▶ σ^2 is a prefix of σ^{21}
- ▶ Execution sequence: m_2m_1

Deterministic State Machine

- ▶ Service implemented as a deterministic state machine
- ▶ Processing requests yield unique state transitions:
 - ▶ $\sigma_1^2 = \sigma_2^2$ and $\sigma_1^{21} = \sigma_2^{21}$.
- ▶ Clients suppress duplicate replies

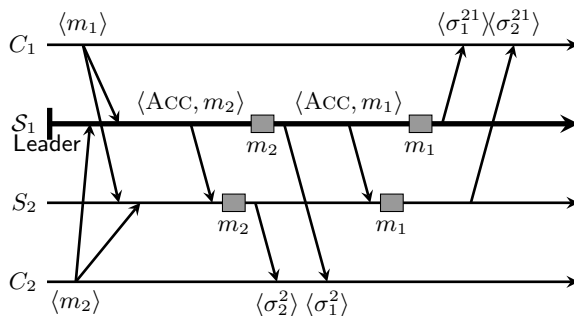
Fault Tolerance with Two Servers: Whoops!



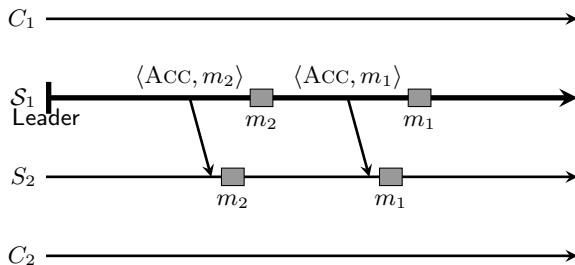
- ▶ Client C_2 sees: $\sigma^2 \sigma^{12}$
 - ▶ σ^2 not a prefix of σ^{12}
- ▶ Client C_1 sees: $\sigma^1 \sigma^{21}$
 - ▶ σ^1 not a prefix of σ^{21}
- ▶ Execution sequence
 - ▶ S_1 : $m_1 m_2$
 - ▶ S_2 : $m_2 m_1$

We Need to Order Client Requests!

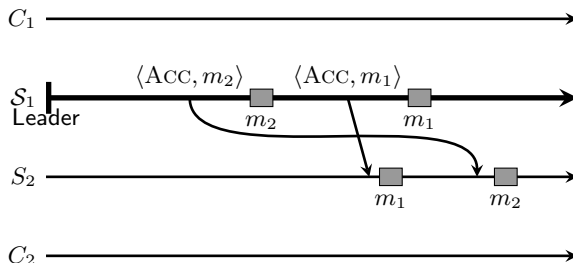
Let's Designate a Leader to Order Requests



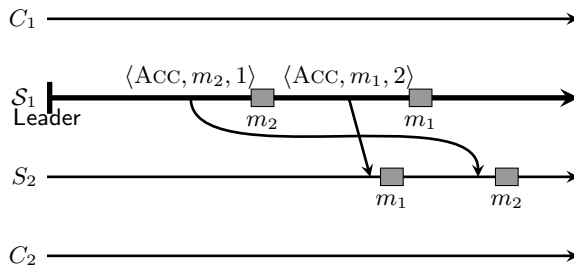
Without Clients



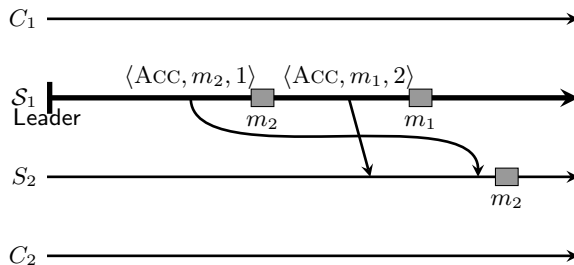
Problem: Also Accept Messages can be Reordered



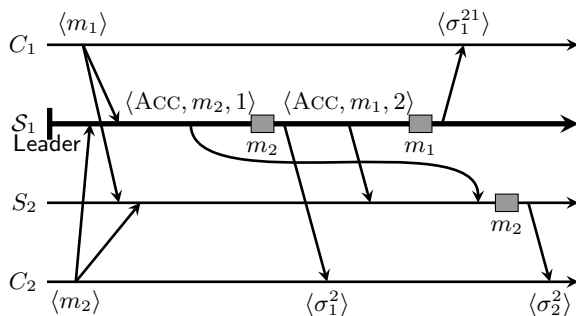
Add Sequence Numbers



Discard Out-of-Order Messages



Now with Clients



Clients Observe The Same Server States as Before

- ▶ Client C_2 sees: σ^2
- ▶ Client C_1 sees: σ^{21}

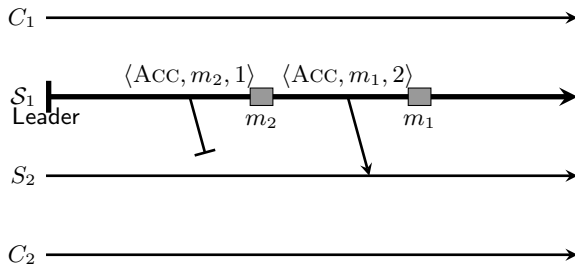
Clients Observe The Same Server States as Before

- ▶ Client C_2 sees: σ^2
- ▶ Client C_1 sees: σ^{21}
- ▶ However, S_2 didn't execute m_1
 - ▶ Q: What to do?

Clients Observe The Same Server States as Before

- ▶ Client C_2 sees: σ^2
- ▶ Client C_1 sees: σ^{21}
- ▶ However, S_2 didn't execute m_1
 - ▶ Q: What to do?
 - ▶ A1: Buffer

Problem: Message Loss – S_2 Won't Execute Anything

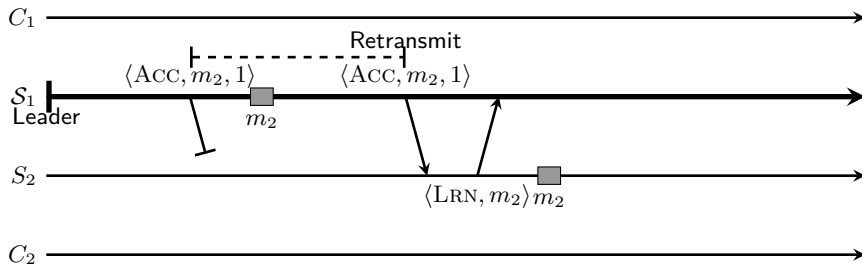


Clients Observe The Same Server States as Before

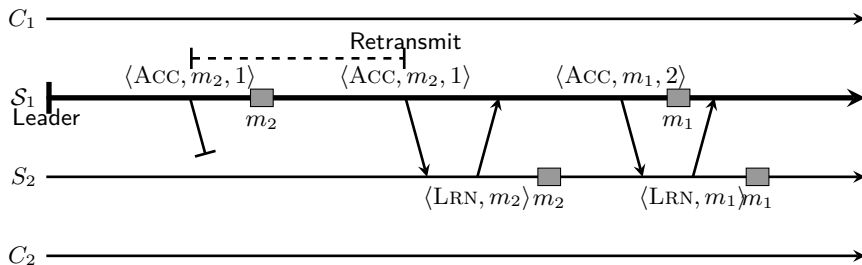
- ▶ Client C_2 sees: σ^2
- ▶ Client C_1 sees: σ^{21}
- ▶ However, S_2 didn't execute m_1
 - ▶ Q: What to do?
 - ▶ A1: Buffer
 - ▶ A2: Retransmission mechanism

We Need a Retransmission Mechanism!

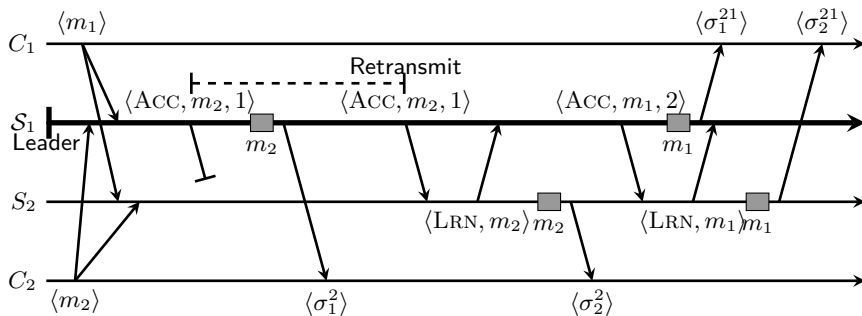
A Learn Stops Retransmission



Don't Send New Accept Until Learn



With Clients



Recap

- ▶ A leader
 - ▶ Decides order of client requests
 - ▶ By sending an accept message to S_2

Recap

- ▶ A leader
 - ▶ Decides order of client requests
 - ▶ By sending an accept message to S_2
- ▶ Sequence numbers
 - ▶ Cope with message reordering

Recap

- ▶ A leader
 - ▶ Decides order of client requests
 - ▶ By sending an accept message to S_2
- ▶ Sequence numbers
 - ▶ Cope with message reordering
- ▶ Retransmission mechanism
 - ▶ Cope with message loss
 - ▶ Leader only sends next accept when learn from S_2

Recap

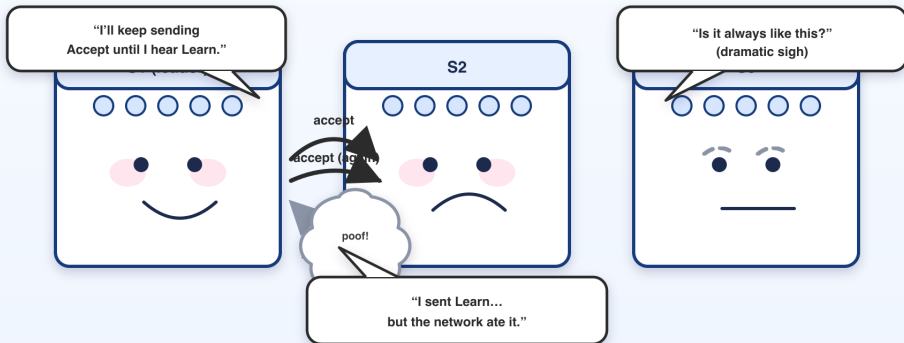
- ▶ A leader
 - ▶ Decides order of client requests
 - ▶ By sending an accept message to S_2
- ▶ Sequence numbers
 - ▶ Cope with message reordering
- ▶ Retransmission mechanism
 - ▶ Cope with message loss
 - ▶ Leader only sends next accept when learn from S_2

Combination of mechanisms:
RetransAccept protocol

RetransAccept

RetransAccept: “Did you get my learn?”

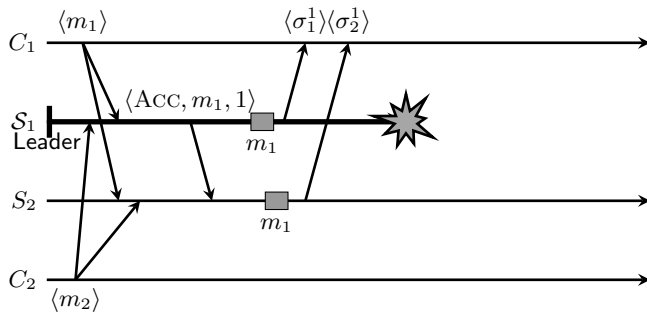
When a Learn gets lost, the leader keeps re-sending Accept until it hears back.



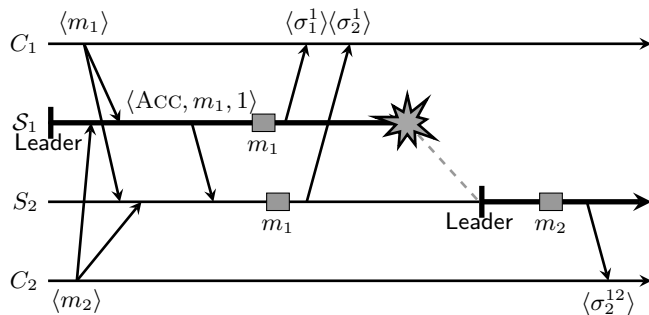
Missing messages are annoying — but retransmission makes progress anyway.

What About Server Crashes?

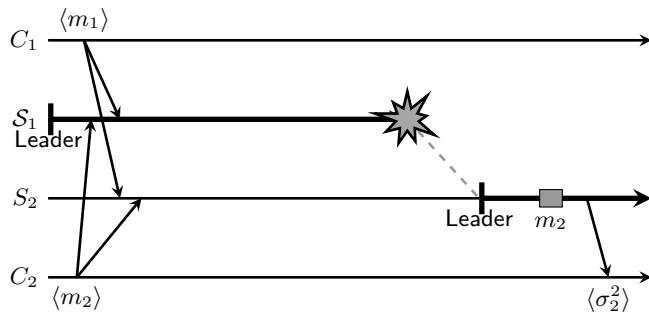
Crash



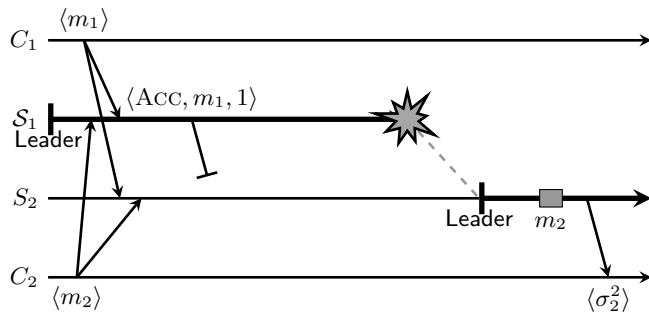
Crash: Leader Takeover



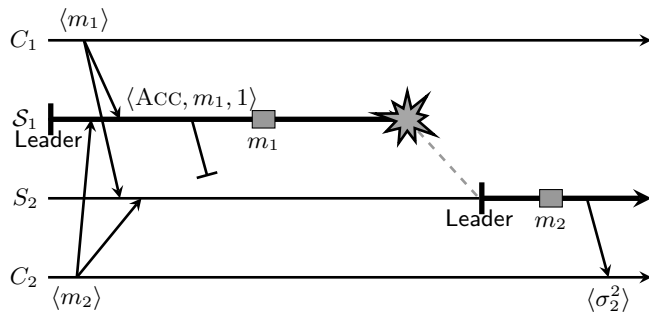
Single Server Rule: Case 1



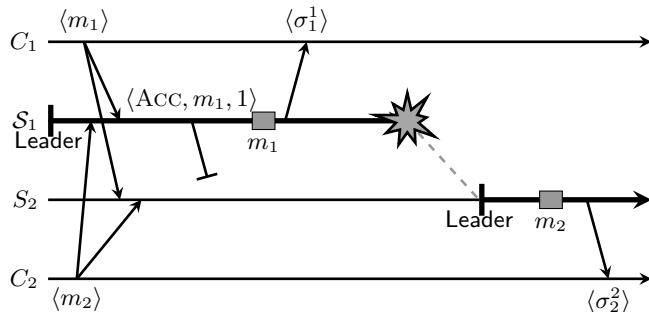
Single Server Rule: Case 2



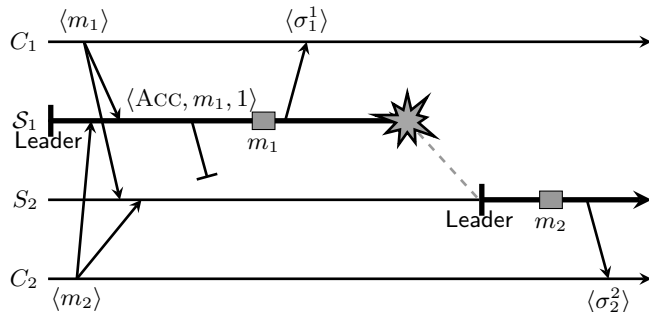
Single Server Rule: Case 3



Single Server Rule: Case 4 – A Problem



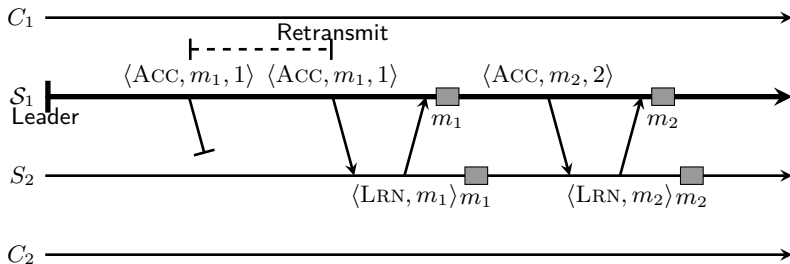
Lock Service: Both clients get the lock



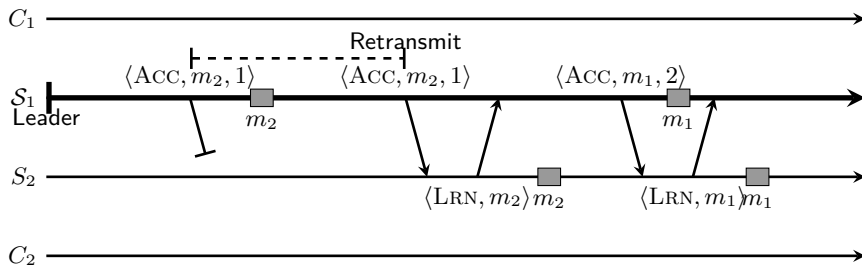
Single Server Rule: Case 4 – A Problem

- ▶ Imagine that (S_1, S_2) is a fault tolerant lock service
- ▶ Both clients could have gotten the lock

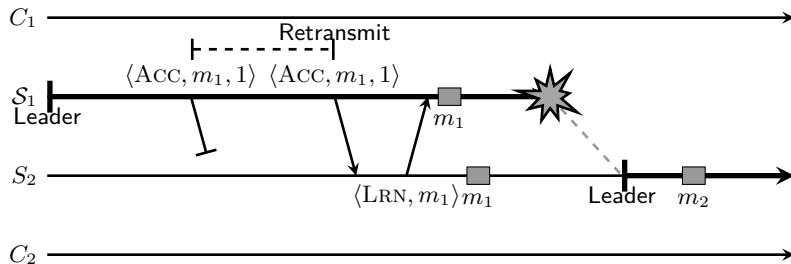
Solution: Leader Waits for Learn Before Executing



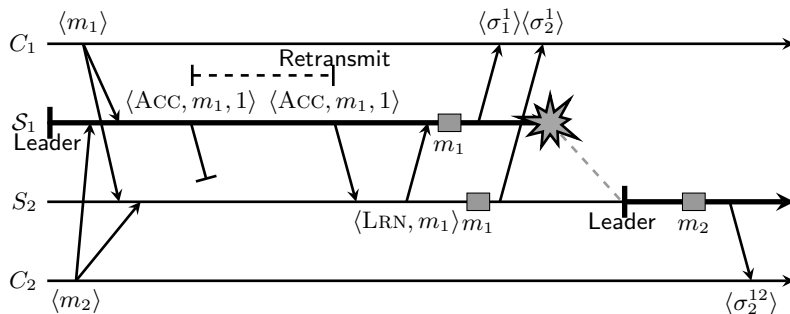
Recall Earlier Version



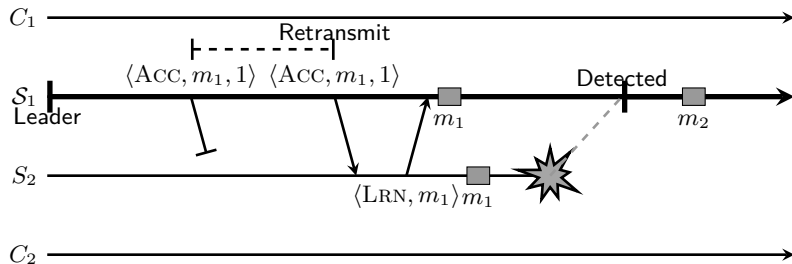
Now Leader Takeover is Safe



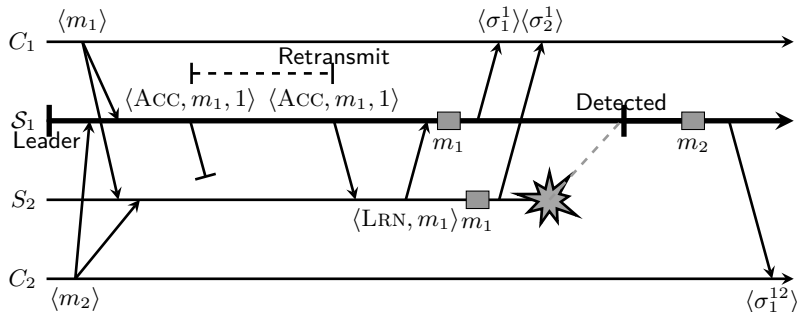
Let's Add Client Messages



Leader Remain in Control when S_2 Crash



Let's Add Client Messages Again



Recap: The Problem

- ▶ When we detect a server crash
 - ▶ Switch to *SingleServer* mode

Recap: The Problem

- ▶ When we detect a server crash
 - ▶ Switch to *SingleServer* mode
- ▶ Problem with *RetransAccept* protocol:
 - ▶ Leader might have replied to a client and then crashed, without ensuring that S_2 saw the accept

Recap: The Problem

- ▶ When we detect a server crash
 - ▶ Switch to *SingleServer* mode
- ▶ Problem with *RetransAccept* protocol:
 - ▶ Leader might have replied to a client and then crashed, without ensuring that S_2 saw the accept
 - ▶ S_2 takes over and may execute a different request in *SingleServer* mode

Recap: WaitForLearn Protocol

- ▶ Leader waits for a learn from S_2

Recap: WaitForLearn Protocol

- ▶ Leader waits for a learn from S_2
- ▶ S_2 can execute after seeing an accept from the leader
 - ▶ Because the accept is also an implicit learn

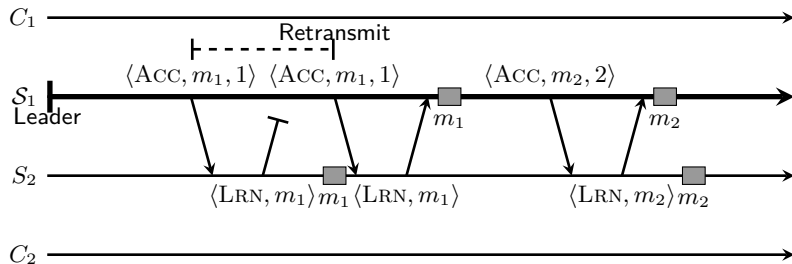
Question

- ▶ Q: What happens if the learn message to the leader is lost?

Question

- ▶ Q: What happens if the learn message to the leader is lost?
- ▶ A: The leader uses *RetransAccept*; the accept will be retransmitted. So no need for another retransmit protocol.

If the Learn is Lost, Retrans will fix it



Somewhat Rougher Road Ahead!

False Detection

- ▶ So far we have assumed that failure detection is accurate

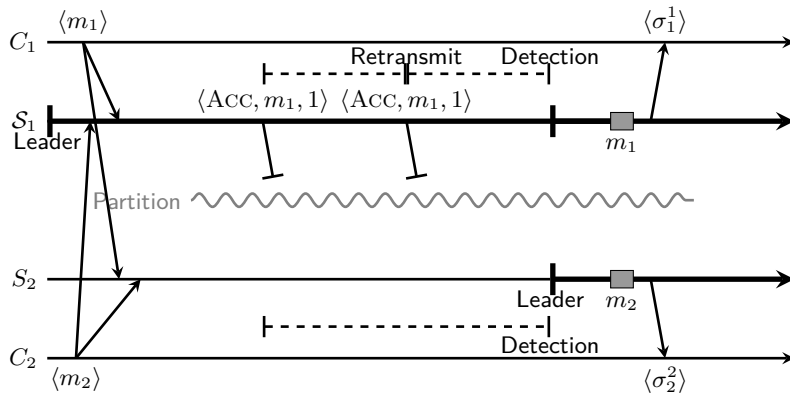
False Detection

- ▶ So far we have assumed that failure detection is accurate
- ▶ But in an asynchronous environment
 - ▶ There is always a chance of false detection
 - ▶ Because it is impossible to pick the right timeout delay

False Detection

- ▶ So far we have assumed that failure detection is accurate
- ▶ But in an asynchronous environment
 - ▶ There is always a chance of false detection
 - ▶ Because it is impossible to pick the right timeout delay
- ▶ We now consider false detection in the context of network partitions

Problem: Network Partitions



Network Partition

- ▶ Each server can switch to *SingleServer* mode (no coordination) and make progress

Network Partition

- ▶ Each server can switch to *SingleServer* mode (no coordination) and make progress
- ▶ But it will lead to inconsistencies
 - ▶ S_1 has state σ^1
 - ▶ S_2 has state σ^2

Network Partition

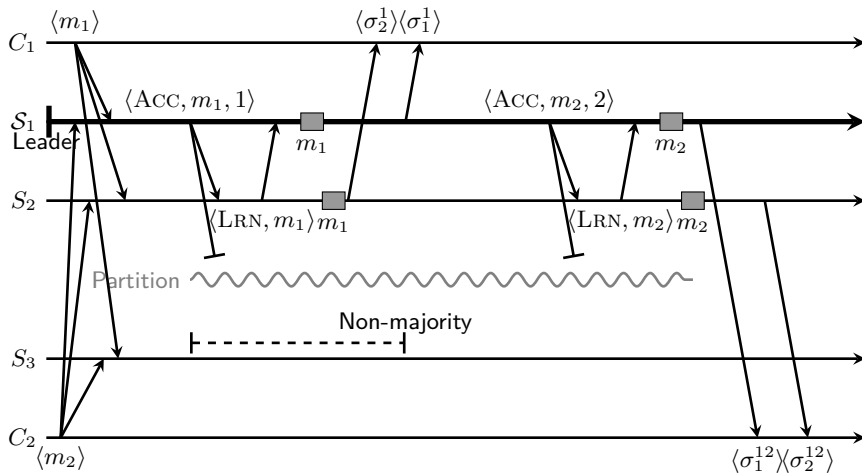
- ▶ Each server can switch to *SingleServer* mode (no coordination) and make progress
- ▶ But it will lead to inconsistencies
 - ▶ S_1 has state σ^1
 - ▶ S_2 has state σ^2
- ▶ Reconciling the state divergence
 - ▶ Involves rollback on multiple clients

Network Partition

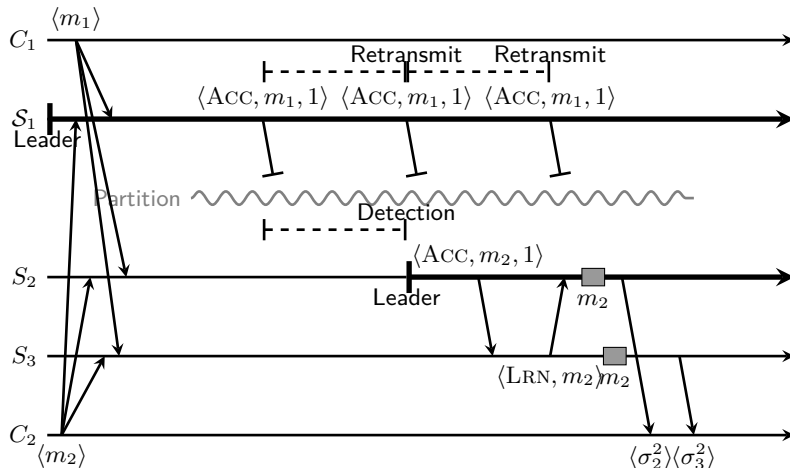
- ▶ Each server can switch to *SingleServer* mode (no coordination) and make progress
- ▶ But it will lead to inconsistencies
 - ▶ S_1 has state σ^1
 - ▶ S_2 has state σ^2
- ▶ Reconciling the state divergence
 - ▶ Involves rollback on multiple clients
 - ▶ Quickly becomes unmanageable

We Want to Avoid Relying on Clients!

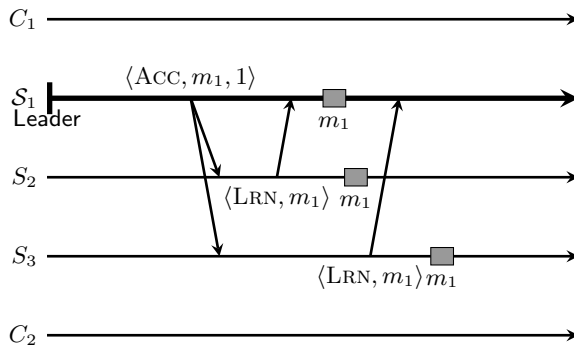
Add Another Server; Make Progress in Majority Partition



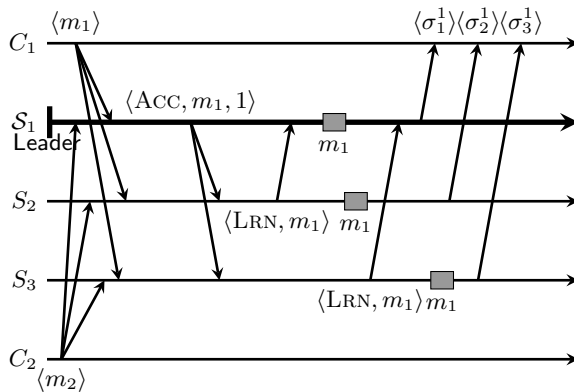
New Leader in Majority Partition



WaitForLearn Without Partition



WaitForLearn With Clients



Recap: Network Partition

- ▶ We added another server, S_3
 - ▶ To avoid rollback using clients

Recap: Network Partition

- ▶ We added another server, S_3
 - ▶ To avoid rollback using clients
- ▶ We still use the *WaitForLearn* protocol
 - ▶ To ensure that another server has seen the accept message

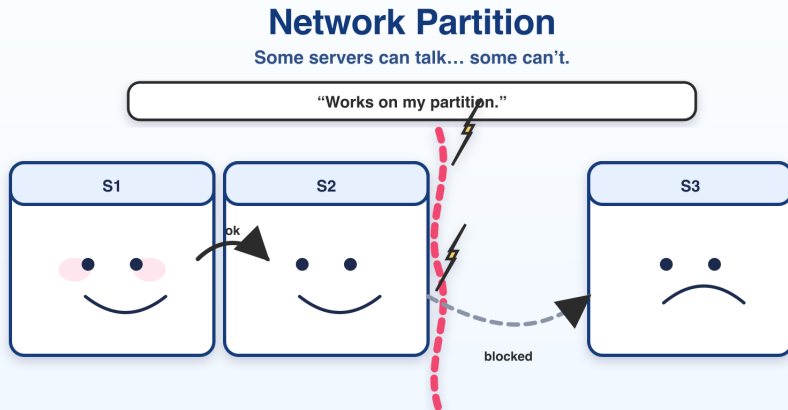
Recap: Network Partition

- ▶ We added another server, S_3
 - ▶ To avoid rollback using clients
- ▶ We still use the *WaitForLearn* protocol
 - ▶ To ensure that another server has seen the accept message
- ▶ Leader only needs to wait for *one* learn before executing the request
 - ▶ Allows the leader to make progress, when one of the other servers has crashed or is temporarily unavailable

Recap: Network Partition

- ▶ We added another server, S_3
 - ▶ To avoid rollback using clients
- ▶ We still use the *WaitForLearn* protocol
 - ▶ To ensure that another server has seen the accept message
- ▶ Leader only needs to wait for *one* learn before executing the request
 - ▶ Allows the leader to make progress, when one of the other servers has crashed or is temporarily unavailable
- ▶ But we still only tolerate one concurrent failure
 - ▶ Either a crash or a network partition

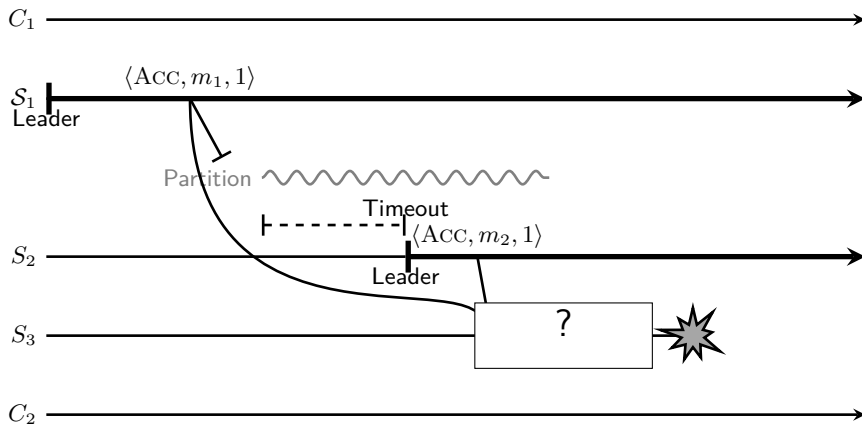
Network Partition



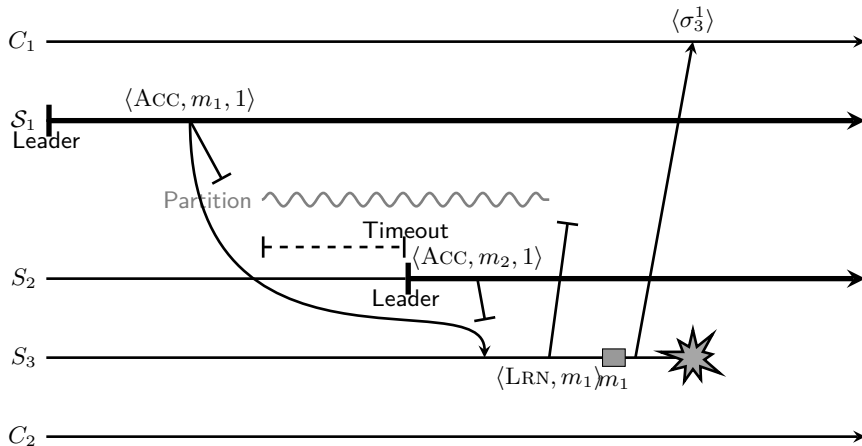
Consensus needs a majority that can still communicate.

What can go Wrong: Concurrent Crash and Partition

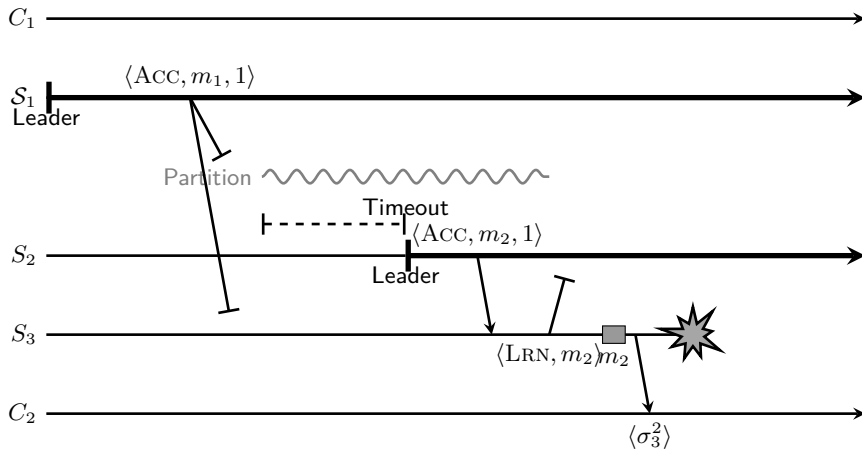
Concurrent Crash and Partition



Crash and Partition: Outcome 1 – m_1 Executed



Crash and Partition: Outcome 2 – m_2 Executed



Recap: Crash and Partition

- ▶ S_3 crashed
 - ▶ But *it could* have executed either m_1 or m_2
 - ▶ And replied to a client

Recap: Crash and Partition

- ▶ S_3 crashed
 - ▶ But *it could* have executed either m_1 or m_2
 - ▶ And replied to a client
- ▶ Other servers cannot determine which message, if any, was executed

Recap: Crash and Partition

- ▶ S_3 crashed
 - ▶ But *it could* have executed either m_1 or m_2
 - ▶ And replied to a client
- ▶ Other servers cannot determine which message, if any, was executed
 - ▶ Maybe we could talk to clients?
 - ▶ We don't want to rely on clients!

Explicit Leader Change Mechanism

- ▶ Above problem is rooted in possibility of false detection
 - ▶ Can lead to several servers thinking they are leaders
 - ▶ And sending accept messages concurrently

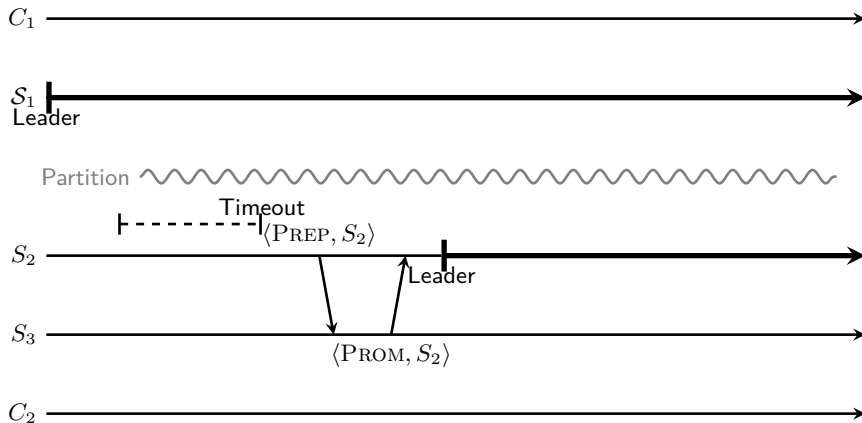
Explicit Leader Change Mechanism

- ▶ Above problem is rooted in possibility of false detection
 - ▶ Can lead to several servers thinking they are leaders
 - ▶ And sending accept messages concurrently
- ▶ It can be solved by an explicit leader takeover protocol

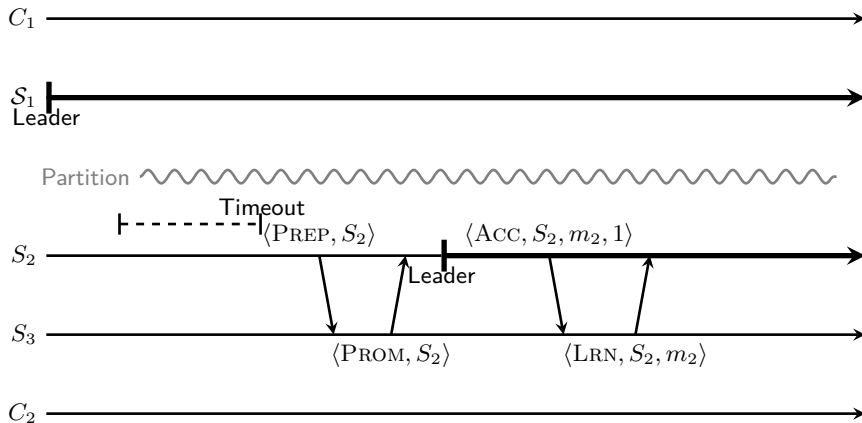
Explicit Leader Change Mechanism

- ▶ Above problem is rooted in possibility of false detection
 - ▶ Can lead to several servers thinking they are leaders
 - ▶ And sending accept messages concurrently
- ▶ It can be solved by an explicit leader takeover protocol
- ▶ We need a way to
 - ▶ Distinguish messages from different leaders
 - ▶ Change the leader

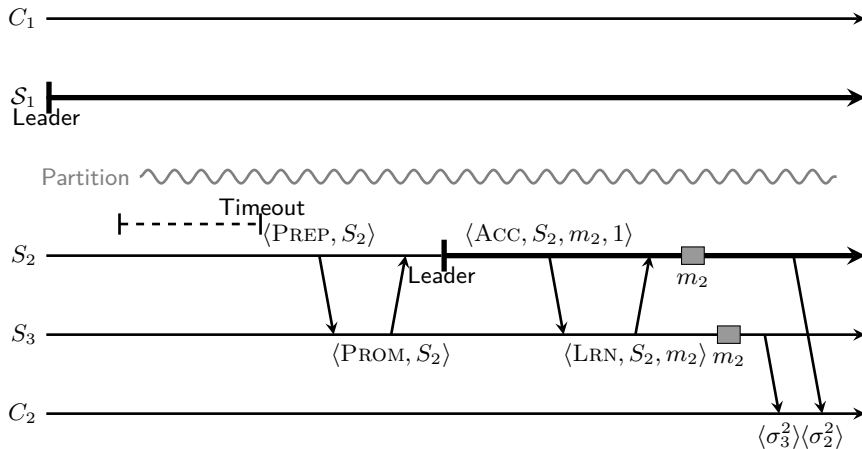
Explicit Leader Change



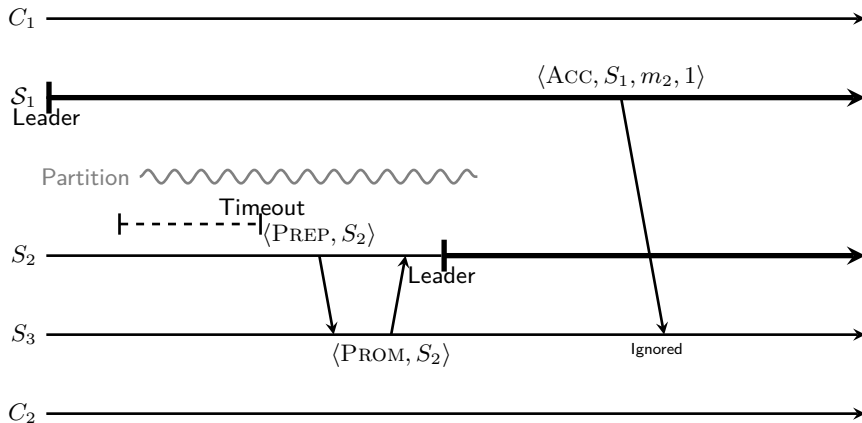
Leader Identifiers in Accept and Learn Messages



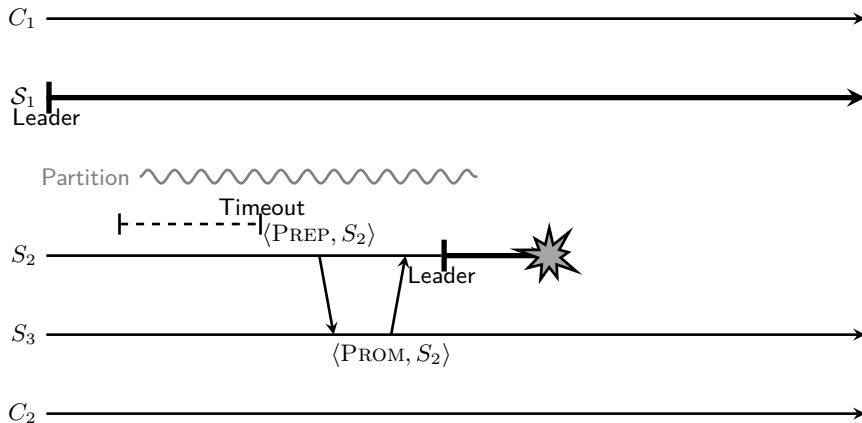
With Client Replies



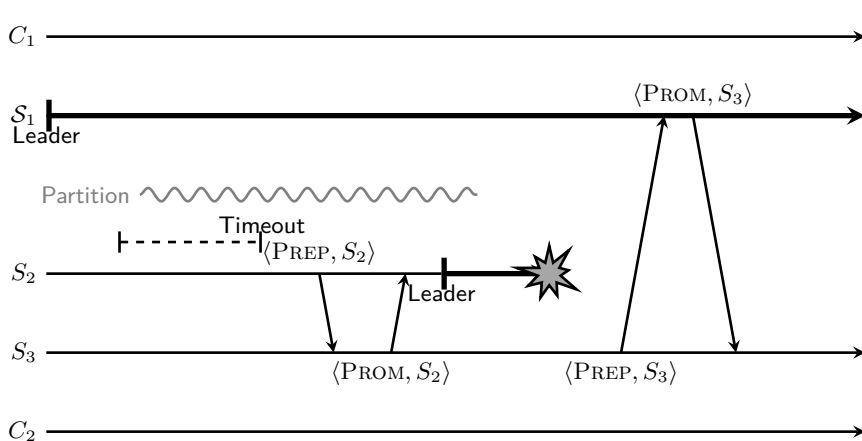
Ignore Accept From Old Leader



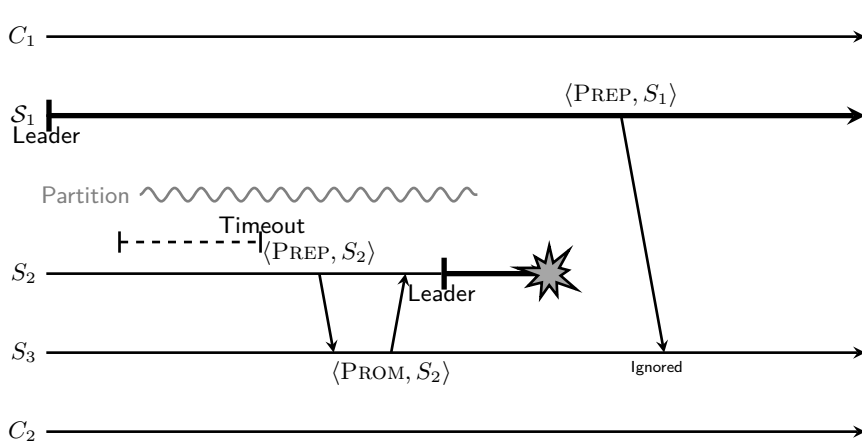
What Happens Now?



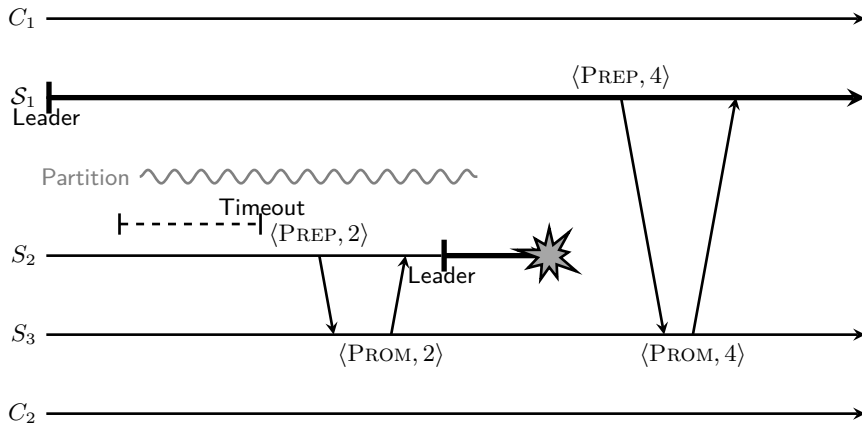
Case 1: S_3 Takes Over?



Case 2: S_1 Takes Over Again?



Replace Leader Identifiers With Round Numbers



Recap: Leader Change

- ▶ Added round number rnd in messages
 - ▶ To identify the leader
 - ▶ $\langle ACC, rnd, m, seqno \rangle$: Sent by leader of round rnd
 - ▶ $\langle LRN, rnd, m \rangle$: Sent to leader of round rnd

Recap: Leader Change

- ▶ Added round number rnd in messages
 - ▶ To identify the leader
 - ▶ $\langle ACC, rnd, m, seqno \rangle$: Sent by leader of round rnd
 - ▶ $\langle LRN, rnd, m \rangle$: Sent to leader of round rnd
 - ▶ Round numbers are assigned:
 - ▶ S_1 : 1, 4, 7, ...
 - ▶ S_2 : 2, 5, 8, ...
 - ▶ S_3 : 3, 6, 9, ...
 - ▶ Skipping rounds is possible

Recap: Leader Change

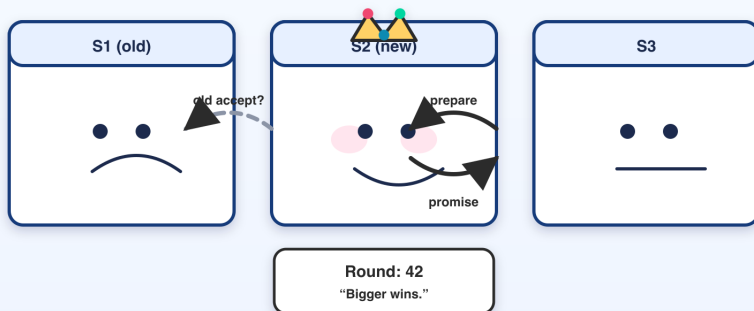
- ▶ Added round number rnd in messages
 - ▶ To identify the leader
 - ▶ $\langle ACC, rnd, m, seqno \rangle$: Sent by leader of round rnd
 - ▶ $\langle LRN, rnd, m \rangle$: Sent to leader of round rnd
 - ▶ Round numbers are assigned:
 - ▶ S_1 : 1, 4, 7, ...
 - ▶ S_2 : 2, 5, 8, ...
 - ▶ S_3 : 3, 6, 9, ...
 - ▶ Skipping rounds is possible
- ▶ Added two new messages
 - ▶ $\langle PREP, rnd \rangle$: Request to become leader for round rnd
 - ▶ $\langle PROM, rnd \rangle$: Promise not to accept messages from a lower round than rnd (i.e. an older leader)

Leader Change

Leader Change

New leader, new round number — but we still keep safety.

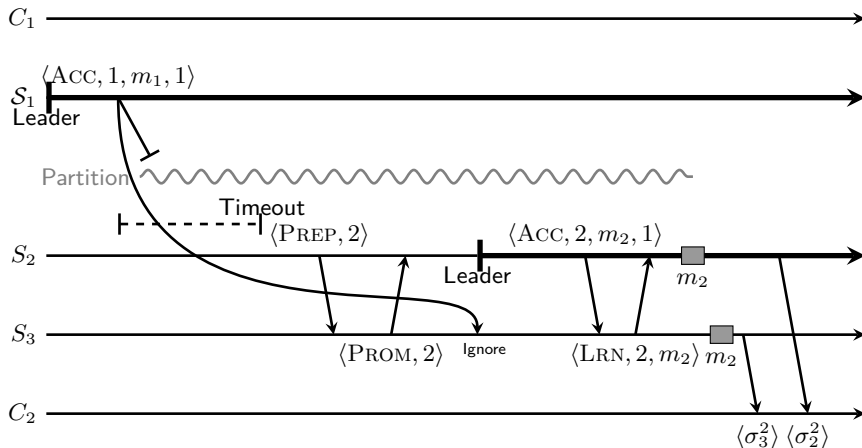
"I brought a bigger round number."



Leader change is a reset — with memory of what was already accepted.

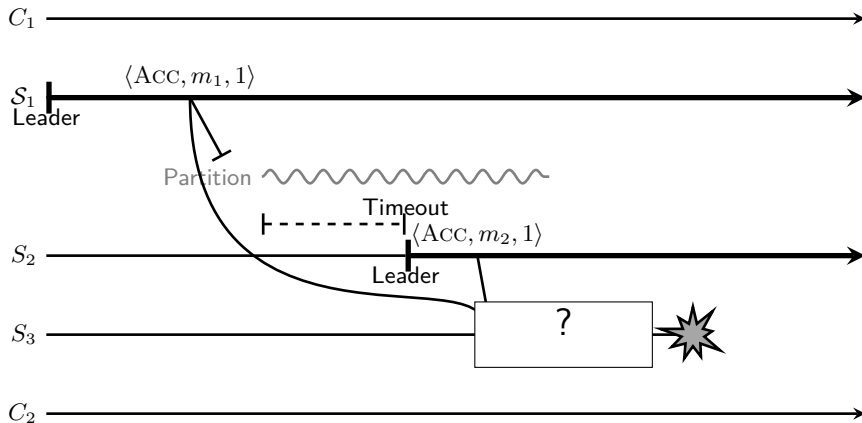
Let's Apply This Together
With Accept and Learn

S_3 Ignores Accept Message From Old Leader

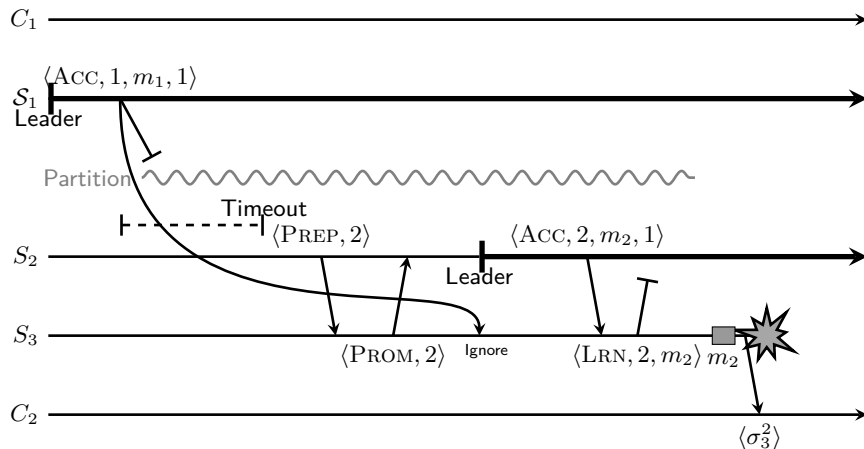


Let's Recall the Problem we are Trying
to Solve

We Don't Know What S_3 Did Before Crashing



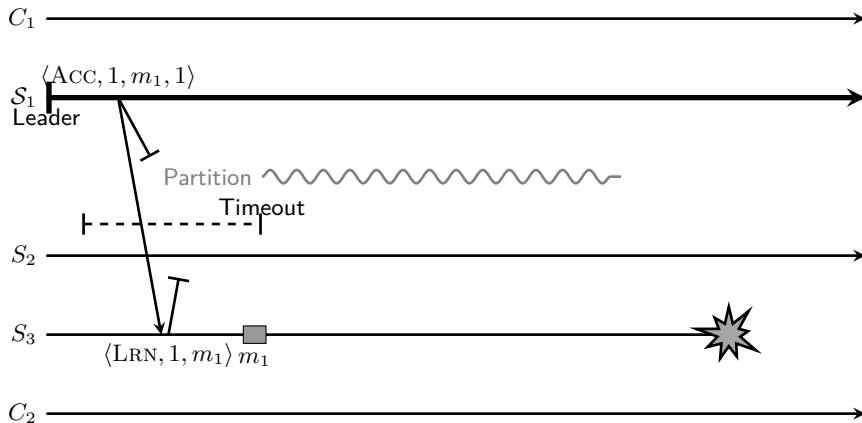
Do We Know Now?



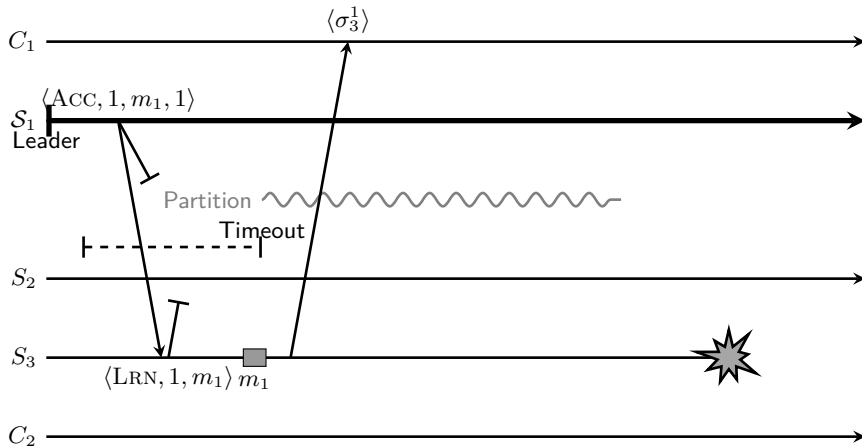
No we don't!

But it is Safe to Continue
as If m_2 Had Been Executed

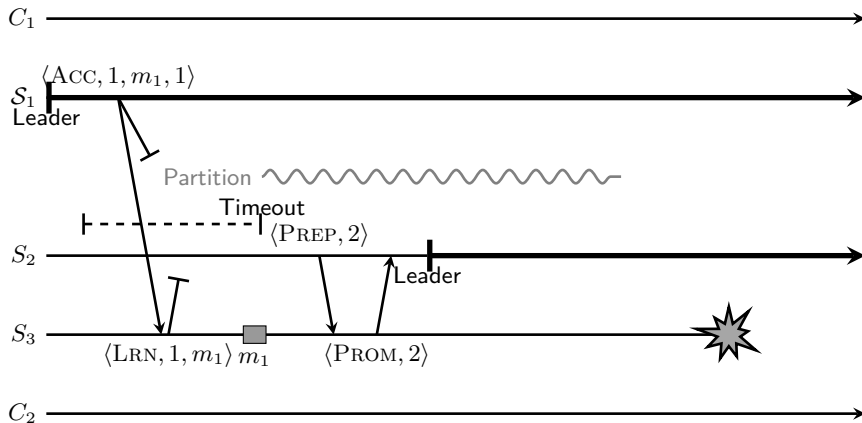
What Happens If S_3 Learn m_1 ?



What Happens If S_3 Learn m_1 ?



Does Leader Change Help?

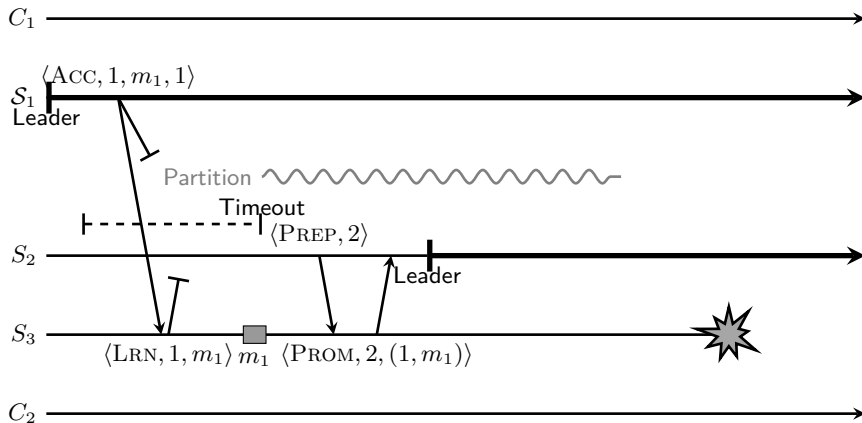


No!

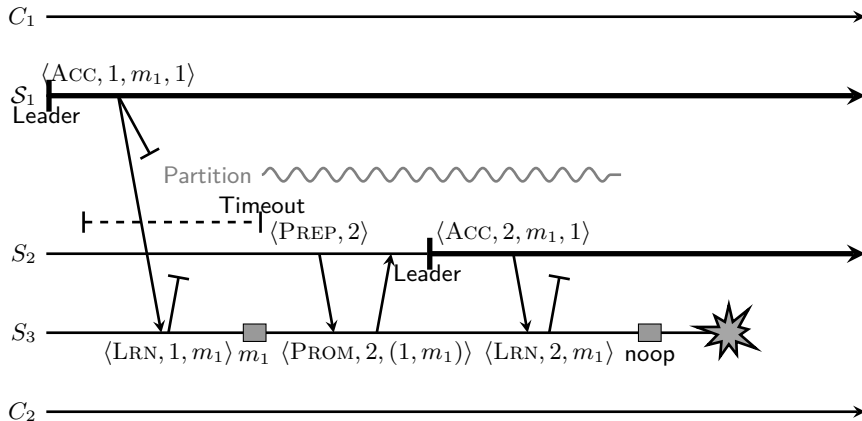
We Still don't Know What
 S_3 Did Before Crashing.

But the fix is Easy!

Tell new Leader About Accepted Messages

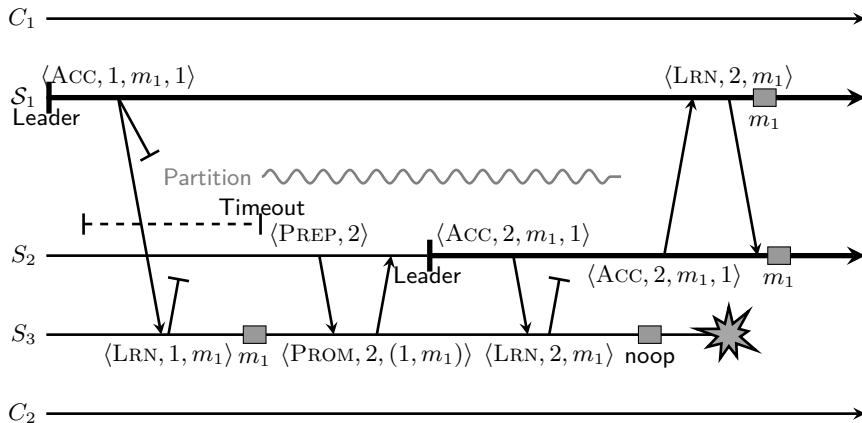


The new Leader Resends Accept for Those Messages

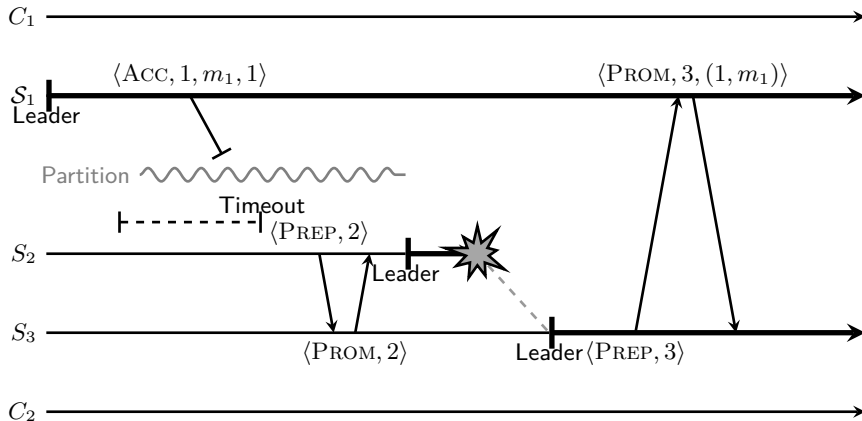


Learn was Lost and S_3 Crashed.
Leader Still can't Execute m_1 .

Leader Also Resends Accept After Merge



Promise from old Leader Includes Accepted Messages



Recap: Leader Change 2

- ▶ Added information about accept from previous leader:
 $\langle \text{PROM}, rnd, (1, m_1) \rangle$
 - ▶ Promise not to accept messages from a lower round than rnd
 - ▶ Previous leader sent m_1 in round 1
 - ▶ Typical naming: $\langle \text{PROM}, rnd, (vrnd, vval) \rangle$

Recap: Leader Change 2

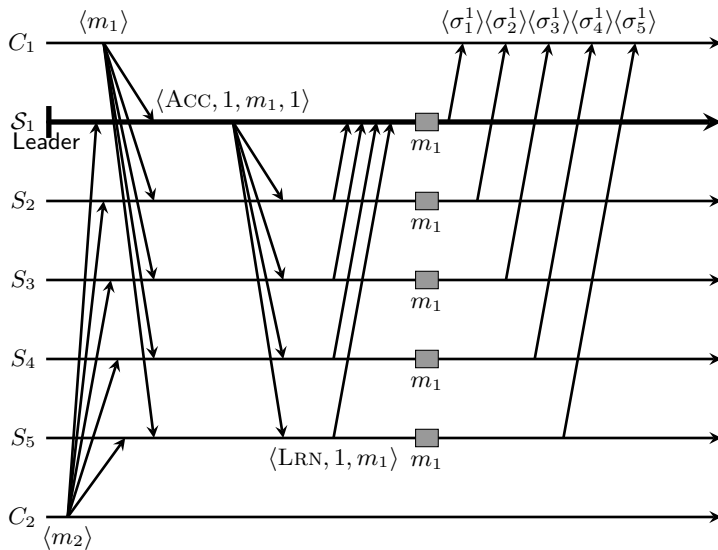
- ▶ Added information about accept from previous leader:
 $\langle \text{PROM}, rnd, (1, m_1) \rangle$
 - ▶ Promise not to accept messages from a lower round than rnd
 - ▶ Previous leader sent m_1 in round 1
 - ▶ Typical naming: $\langle \text{PROM}, rnd, (vrnd, vval) \rangle$
- ▶ Leader resends accept for messages identified in the promise
 - ▶ After receiving the promise
 - ▶ After a partition merge

What About More Than one Crash?

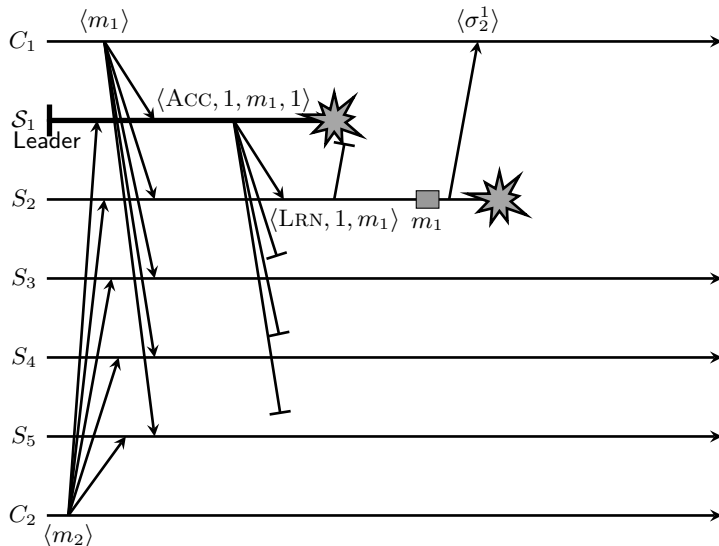
What About More Than one Crash?

- ▶ Increase the number of servers
- ▶ Limit progress to a majority partition:
 - ▶ Can only tolerate that fewer than half of the servers fail
 - ▶ To tolerate f crashes, we need $2f + 1$
 - ▶ Majority: $f + 1$

With Five Servers



With Five Servers, S_2 Cannot Execute After Accept



With Five Servers, S_2 Cannot Execute After Accept

- ▶ A combination of message loss and crashes
 - ▶ Prevent non-leader servers from executing after receiving an accept

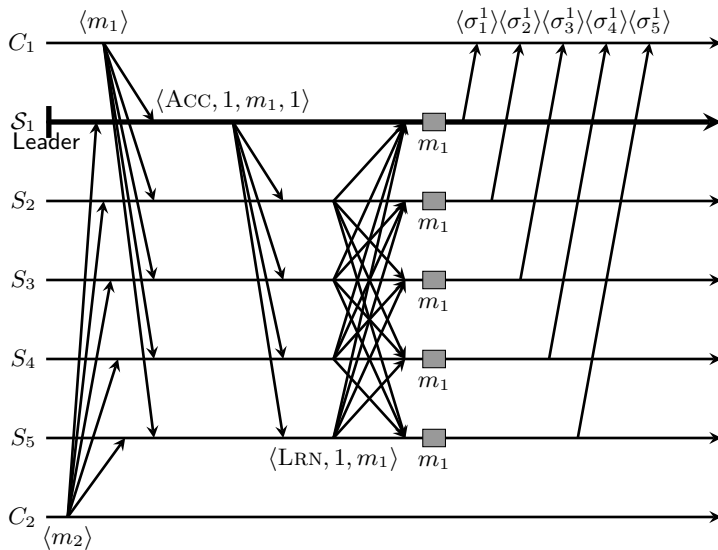
With Five Servers, S_2 Cannot Execute After Accept

- ▶ A combination of message loss and crashes
 - ▶ Prevent non-leader servers from executing after receiving an accept
 - ▶ This was not necessary for the three server case
 - ▶ The accept from the leader is an implicit learn
 - ▶ And together with its own "learn", can execute!

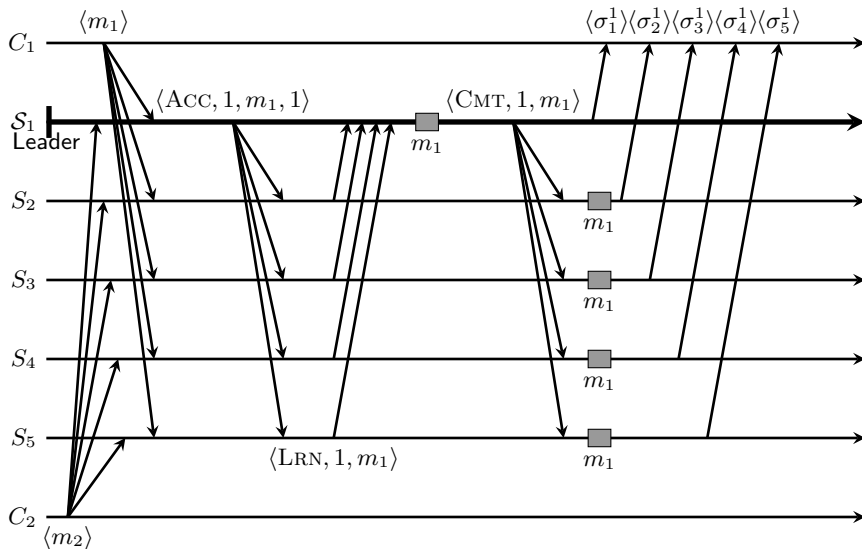
With Five Servers, S_2 Cannot Execute After Accept

- ▶ A combination of message loss and crashes
 - ▶ Prevent non-leader servers from executing after receiving an accept
 - ▶ This was not necessary for the three server case
 - ▶ The accept from the leader is an implicit learn
 - ▶ And together with its own "learn", can execute!
- ▶ There are two solutions:
 - ▶ Wait for all-to-all learn
 - ▶ Wait for commit from leader

All-to-All Learn Before Execute

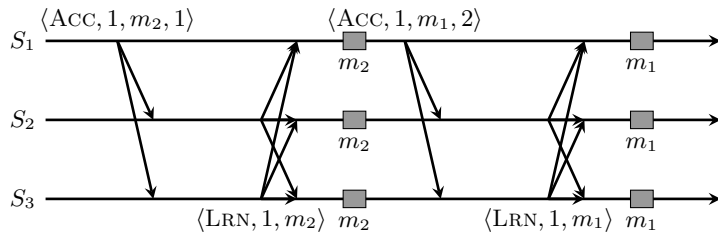


Await Commit Before Execute

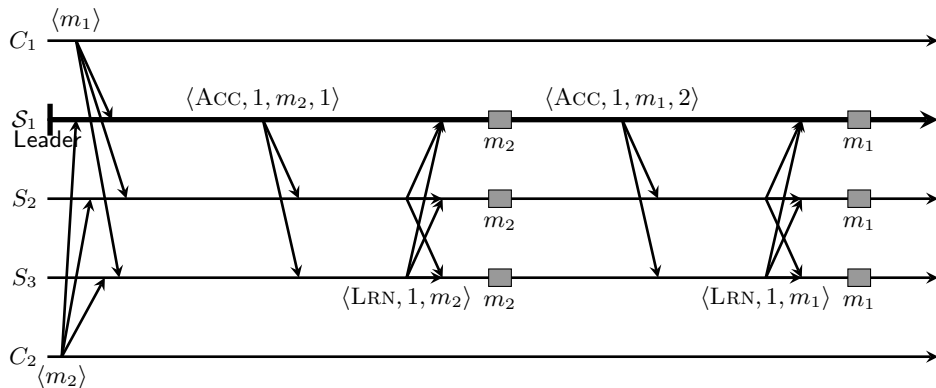


Wrapping it up!

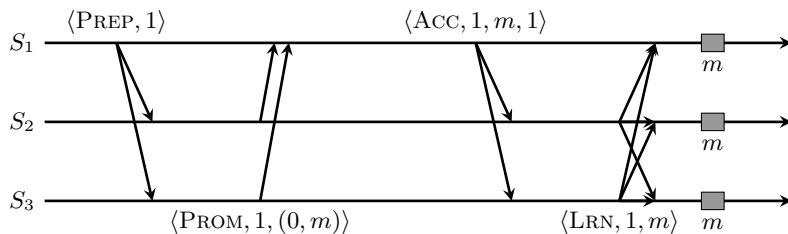
Multi-Paxos



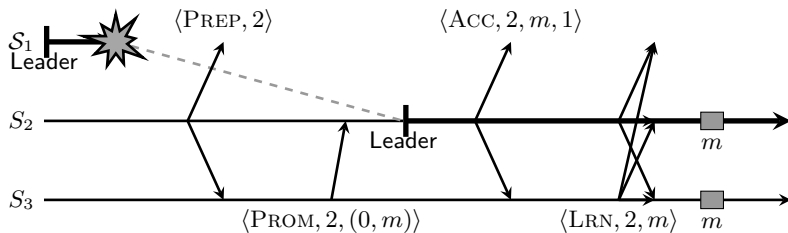
Multi-Paxos with Clients



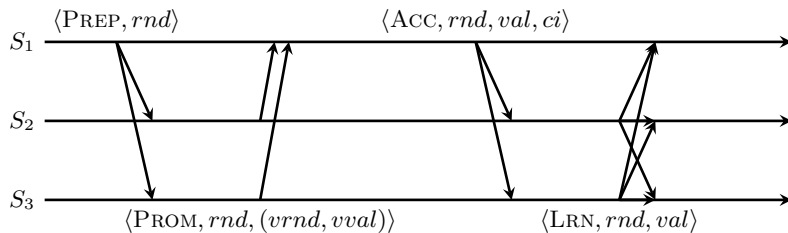
Paxos



Paxos with Failure



Paxos



That's It! Thank You!